VerifyBench: Benchmarking Identity Verification APIs

Case study for Bureau, Product Analyst Intern

Oct 8, 2026 · @Anisha

Why this case study

Bureau runs 120+ API services, and the Product Analyst Intern has to say which ones are winning and which are slipping. This case study shows how I would do that, using a simulated month of verification API traffic, SQL, and charts.

It covers the four parts of the role:

1. **Performance benchmarking:** latency and success rate per API against internal standards and an external competitor.
2. **Data-driven strategy:** why verifications fail, and which segment to fix first to lift conversion.
3. **Foundation and data quality:** an OCR data sourcing tracker for document types, volume and tagging.
4. **Impact reporting:** specific upgrades recommended to Engineering and Product.

All data here is simulated by me to show the method. It is not Bureau data, and the numbers are illustrative.

Benchmarking framework

Every API gets the same five checks, each against an internal standard, so a slipping API shows up the same week.

Metric	How it is measured	Internal standard (assumed)	Why it matters
p50 and p95 latency (ms)	Percentile of successful-call response time, per API per week	p95 under the API's own target (300 ms to 4,000 ms by API)	p95 is what slow users feel; averages hide the tail
Success rate (%)	Successful calls / all calls	97% or higher	A drop here is lost onboarding, not just an error

Metric	How it is measured	Internal standard (assumed)	Why it matters
Week-on-week change (pts)	This week's success rate minus last week's	Alert on a fall of 3 pts or more	Catches sudden drops like an upstream outage
Coverage gaps	Success rate cut by document type, language, device and region	No cut more than 5 pts below the API's average	Finds users the product quietly fails
Competitor gap	Our p95 minus a rival vendor's published or tested p95	Our p95 at or below theirs	Shows where we are best-in-class and where we are not

The competitor figure would come from sandbox tests or published SLAs; here it is simulated.

Worked example: benchmarking 10 APIs

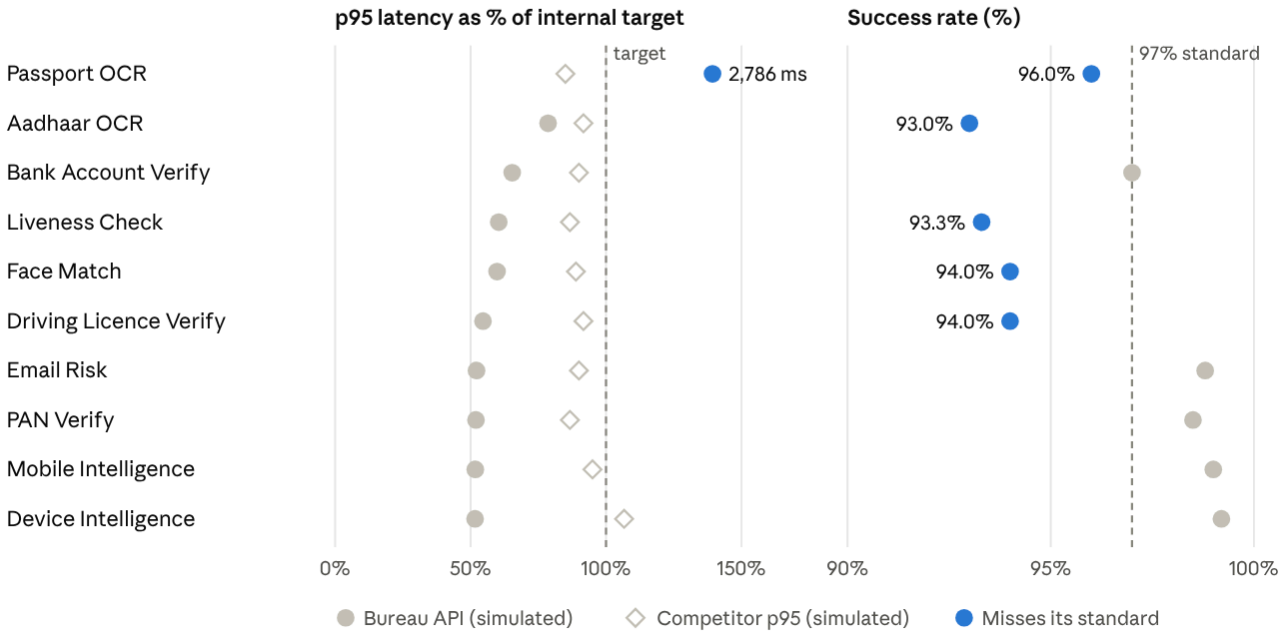
Passport OCR is the only API over its latency target (p95 2,786 ms vs 2,000 ms, and 1,086 ms slower than the competitor). Five APIs fall below the 97% success standard, led by Aadhaar OCR at 93.0%.

I simulated 200,000 calls across 10 verification APIs for September 2026, with four problems planted to test whether the method finds them. It found all four. SQLite has no percentile function, so p95 comes from a window function:

```
WITH ranked AS (
  SELECT api, latency_ms,
         ROW_NUMBER() OVER (PARTITION BY api ORDER BY latency_ms) AS rn,
         COUNT(*)     OVER (PARTITION BY api) AS n
  FROM api_calls WHERE status = 'success'
)
SELECT r.api,
       MAX(CASE WHEN rn = CAST(0.50 * n AS INT) THEN latency_ms END) AS
p50_ms,
       MAX(CASE WHEN rn = CAST(0.95 * n AS INT) THEN latency_ms END) AS
p95_ms,
       s.target_p95_ms, s.competitor_p95_ms
FROM ranked r JOIN standards s USING (api)
GROUP BY r.api
ORDER BY p95_ms * 1.0 / s.target_p95_ms DESC;
```

Passport OCR breaks its latency target; five APIs miss 97% success

10 APIs, simulated September 2026 traffic; 5 below the success standard



Simulated data · 200,000 API calls across 10 APIs, 1 to 30 Sep 2026 · SQLite

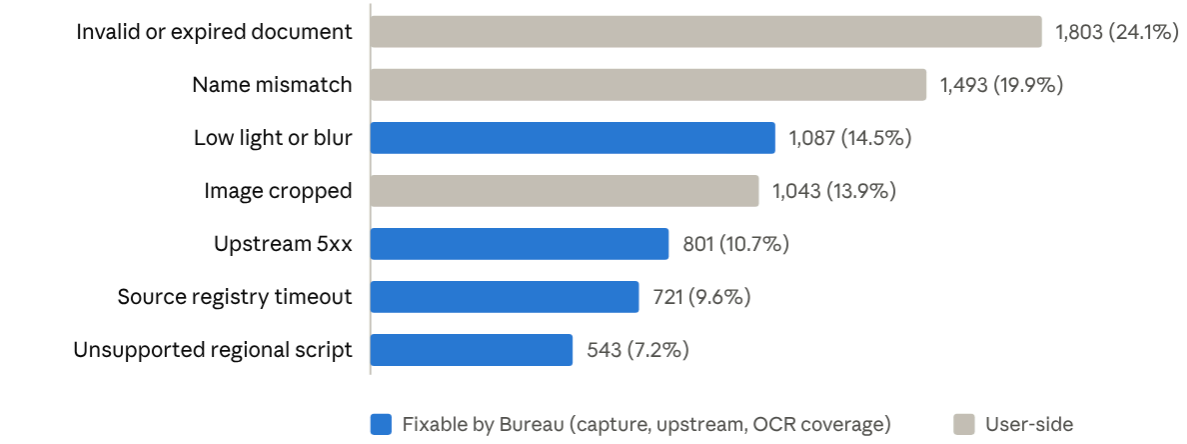
Every other API beats both its target and the competitor on latency, so the latency fix is narrow: one API's slow tail. The success-rate problem is wider, and the next section explains it.

Why did this verification fail?

Three fixable causes cost about 1,520 completed onboardings in the month, and low-end Android face and liveness checks are the largest at about 700.

Four in ten failed verifications come from causes Bureau can fix

7,491 failed calls; 42% from capture, upstream or OCR coverage



Simulated data · 7,491 failed calls, 1 to 30 Sep 2026

A failed verification matters because users who fail complete onboarding 21% of the time, against 88% for users who pass. So each recovered failure is worth about 0.67 completed onboardings. To find where the failures cluster, I cut success rate by segment and compared each segment with the rest of that API's traffic:

```
SELECT channel || ' ' || device_tier       AS segment,
       COUNT(*)                            AS calls,
       ROUND(100.0 * AVG(status = 'success'), 1) AS success_pct,
       ROUND(100.0 * AVG(completed), 1)          AS conversion_pct
FROM api_calls
WHERE api IN ('Face Match', 'Liveness Check')
GROUP BY segment
ORDER BY conversion_pct;
```

Segment that fails	Success rate in segment vs rest	Calls	Completions lost	Likely root cause
Face Match and Liveness on low-end Android	85.2% vs 96.2%	9,550	~700	Low light and blur from weak phone cameras
Driving Licence Verify, 15 to 21 Sep	82.1% vs 97.5%	4,561	~470	Government source registry timing out
Aadhaar OCR on Tamil and Bengali cards	82.3% vs 95.6%	3,989	~350	OCR model not trained on these scripts

Client segment (Lending, Fintech, Gaming, E-commerce) made no difference: all four sit at 96.2% to 96.3% success. The problem lives in the device, the document and the upstream source, not the client.

OCR data sourcing engine

The Aadhaar OCR failures point straight to a sourcing gap: regional-script cards are under-represented in training data, so the fix starts with sourcing, not the model.

I would run sourcing from one tracker, with a volume target per document type and variant, and tag every image the same way. The volumes below are an illustrative plan, not real counts.

Document type and variant	Target images	Collected	Tagged and QA-passed	Gap to target	Next action
Aadhaar, Tamil script	3,000	650	520	2,480	Source first: drives 14% failure on these cards
Aadhaar, Bengali script	3,000	700	560	2,440	Source first, same cause
Passport, older booklet layout	2,000	1,100	950	1,050	Source next: check if it drives the slow p95 tail
Driving licence, state formats	4,000	2,900	2,600	1,400	Fill states with under 100 images
PAN, all layouts	2,000	2,100	1,980	20	Hold; volume is enough
Aadhaar, English and Hindi	4,000	4,300	4,050	0	Hold; sample only for drift checks

Each image gets the same tags, so a training set can be cut by any of them:

- **Document:** type, variant, script or language, issuing state
- **Capture:** device tier, lighting (good, low), blur, crop, angle
- **Ground truth:** the correct value for every field the OCR model reads
- **QA:** a second tagger checks a 10% sample; a batch below 98% field accuracy goes back for re-tagging

A weekly check then compares OCR success by tag. A tag value that drops more than 5 pts below average becomes next week's sourcing priority.

Recommendations for Engineering and Product

Fixing the three failure clusters would recover about 1,520 completed onboardings a month in this simulation, and the capture fix alone is nearly half of that.

Priority	Upgrade	Owner	Expected impact (simulated)
1	Add an on-device light and blur check before Face Match and Liveness upload, with a retake prompt on low-end Android	Engineering (capture SDK)	~700 completions a month

Priority	Upgrade	Owner	Expected impact (simulated)
2	Alert when any API's weekly success rate falls 3 pts; add retry with backoff and a fallback source for Driving Licence registry timeouts	Engineering (platform)	~470 completions in an outage week
3	Source and tag Tamil and Bengali Aadhaar cards, then retrain the OCR model	Product (data)	~350 completions a month
4	Profile Passport OCR's slow tail by document layout: 11.4% of successful calls exceed the 2,000 ms target	Engineering (OCR)	Close the 1,086 ms gap to the competitor
5	Publish one weekly benchmark view for all 120+ APIs using the five checks above	Product analytics	Catch the next drop in days, not weeks

Limits of this study: the data is simulated with four planted problems, competitor figures are invented, and conversion is modelled from two average rates. On real Bureau data, the same queries would run unchanged; only the standards and competitor figures would need real inputs.

About me

I'm Anisha Tiwary, a B.Tech Electronics and Computer Science Engineering graduate from KIIT University, based in Bengaluru and available to join immediately. I work in SQL and Python every day, and my projects cover the same ground as this study: OCR data quality, identity verification failure analysis, SLA benchmarking and risk decisioning.

- Portfolio: anisha-tiwaryy.github.io
- GitHub: github.com/Anisha-Tiwaryy
- Email: anishadgp04@gmail.com, Phone: +91-7602400340